

« Avant d'écrire une ligne de code, sache exactement
ce que la dernière ligne doit avoir accompli. »

CE DOCUMENT EST UN PROTOCOLE.

Applique-le dans l'ordre. Ne saute pas d'étapes.

Chaque section sautée est une dette que tu paieras
en débogage, en reprise, ou devant le client.

PHASE 0 – AVANT D'OUVRIR L'ÉDITEUR

"Penser avant de coder n'est pas de la procrastination.
C'est de l'ingénierie."

0.1 DÉFINIR LE PROBLÈME EN UNE PHRASE

Compléter cette phrase avant tout :

"Ce script doit [ACTION] à partir de [INPUT]
pour produire [OUTPUT] afin que [BÉNÉFICE MÉTIER]."

Exemple :

"Ce script doit envoyer un email de résumé à partir
des ventes du jour dans la base SQL pour que le
directeur commercial commence sa journée informé."

→ Si tu ne peux pas remplir cette phrase, tu n'es
pas prêt à coder. Retourne au client.

0.2 DÉCOMPOSER EN SOUS-PROBLÈMES (Divide & Conquer)

Découper le problème en blocs indépendants et séquentiels.
Chaque bloc = une fonction ou un module.

Méthode :

1. Écrire le flux en français, étape par étape
2. Identifier les entrées et sorties de chaque étape
3. Repérer les étapes qui peuvent échouer
4. Repérer les étapes qui peuvent être réutilisées

Exemple de décomposition :

[LIRE] → [VALIDER] → [TRANSFORMER] → [ÉCRIRE] → [NOTIFIER]

0.3 IDENTIFIER LES INCONNUES AVANT DE COMMENCER

- Quelles données d'entrée sont garanties / incertaines ?
- Quels systèmes externes sont impliqués ?
- Quels sont les cas limites prévisibles ?
(fichier vide, connexion impossible, format inattendu)
- Ce script tournera-t-il une fois ou en production ?

0.4 CHOISIR LE NIVEAU DE COMPLEXITÉ ADAPTÉ

Ne pas sur-architecturer. Poser la question :

Script one-shot	→ script linéaire simple, < 100 lignes
Script récurrent	→ fonctions, logging, gestion d'erreurs
Outil interne	→ classes, config externe, tests
Produit / API	→ architecture complète, CI/CD, docs

→ Commencer toujours au niveau inférieur.
Monter en complexité si le besoin est prouvé,
jamais par anticipation.

PHASE 1 – MISE EN PLACE DE L'ENVIRONNEMENT

1.1 STRUCTURE DE PROJET STANDARD

projet/

```

├── main.py                ← point d'entrée unique
├── config.py              ← constantes et paramètres
├── .env                   ← credentials (jamais sur git)
├── .env.example           ← template sans valeurs réelles
├── .gitignore             ← inclure .env, data/, logs/
├── requirements.txt        ← dépendances figées avec versions
├── README.md              ← comment installer et lancer
├── src/                   ← modules métier
│   ├── __init__.py
│   ├── extract.py
│   ├── transform.py
│   ├── load.py
│   └── notify.py
├── tests/                 ← tests unitaires
│   ├── __init__.py
│   └── test_transform.py
├── logs/                  ← générés à l'exécution
└── data/
    ├── raw/               ← jamais modifié
    ├── interim/
    └── output/

```

RÈGLE : Adapter la structure au niveau de complexité (Section 0.4). Un one-shot n'a pas besoin de src/.

1.2 ENVIRONNEMENT VIRTUEL – TOUJOURS

```

python -m venv venv
source venv/bin/activate      # Linux/Mac
venv\Scripts\activate        # Windows
pip install -r requirements.txt

```

→ Ne jamais installer en global.
Chaque projet a ses propres dépendances isolées.

1.3 GESTION DES CREDENTIALS

Utiliser python-dotenv :

```

# .env
DB_HOST=localhost
DB_PASSWORD=secret
API_KEY=abc123

# Dans le code
from dotenv import load_dotenv
import os
load_dotenv()
db_password = os.getenv("DB_PASSWORD")

```

RÈGLES ABSOLUES :

→ .env dans .gitignore. Toujours. Sans exception.
→ .env.example dans le repo avec des valeurs fictives
→ Jamais de credential en dur dans le code
→ Jamais de credential dans un print() ou un log

1.4 requirements.txt – FIGER LES VERSIONS

pip freeze > requirements.txt

Exemple de contenu :

```

pandas==2.1.0
requests==2.31.0
python-dotenv==1.0.0
loguru==0.7.2

```

→ Figier les versions évite les régressions lors d'une réinstallation chez le client ou en prod.

PHASE 2 – ARCHITECTURE DU CODE

"La structure du code est la première documentation."

2.1 ANATOMIE D'UN SCRIPT D'AUTOMATISATION ROBUSTE

main.py – structure type

```
# 1. IMPORTS
import os, sys
from dotenv import load_dotenv
from loguru import logger
from src.extract import extract_data
from src.transform import clean_data
from src.load import save_results

# 2. CONFIGURATION
load_dotenv()
LOG_FILE = "logs/run_{time}.log"
logger.add(LOG_FILE, rotation="1 week")

# 3. FONCTION PRINCIPALE
def main():
    logger.info("=== Démarrage du pipeline ===")
    try:
        data = extract_data()
        logger.info(f"{len(data)} lignes extraites")

        clean = clean_data(data)
        logger.info(f"{len(clean)} lignes après nettoyage")

        save_results(clean)
        logger.success("Pipeline terminé avec succès")

    except Exception as e:
        logger.error(f"Échec du pipeline : {e}")
        sys.exit(1)

# 4. POINT D'ENTRÉE
if __name__ == "__main__":
    main()
```

2.2 RÈGLES DE CONCEPTION DES FONCTIONS

UNE FONCTION = UNE RESPONSABILITÉ

→ Si tu utilises "et" pour décrire ce que fait une fonction, elle en fait trop.

SIGNATURE CLAIRE

```
def process_orders(df: pd.DataFrame,
                  min_amount: float = 0.0
                  ) -> pd.DataFrame:
```

- Typer les arguments et le retour (type hints)
- Valeurs par défaut pour les paramètres optionnels
- Jamais de variable globale modifiée dans une fonction

DOCSTRING MINIMALE POUR TOUT CE QUI VA EN PROD

```
def clean_dates(df):
    """
    Normalise la colonne 'date' en datetime UTC.
    Supprime les lignes avec dates invalides.

    Args:
        df: DataFrame avec colonne 'date' en string

    Returns:
        DataFrame avec colonne 'date' en datetime
    """
```

TAILLE MAXIMALE

- Si une fonction dépasse 30 lignes, la découper.
- Si un fichier dépasse 200 lignes, le diviser.

2.3 GESTION DES CONFIGURATIONS

```
# config.py
import os
from dotenv import load_dotenv
load_dotenv()

# Chemins
DATA_RAW = "data/raw/"
DATA_OUTPUT = "data/output/"

# Paramètres métier
BATCH_SIZE = 1000
RETRY_MAX = 3
TIMEOUT_SEC = 30

# Credentials via env
DB_HOST = os.getenv("DB_HOST")
API_KEY = os.getenv("API_KEY")

→ Centraliser tous les paramètres dans config.py
Jamais de valeur magique dispersée dans le code.
```

2.4 PATTERNS COURANTS EN AUTOMATISATION

RETRY AVEC BACKOFF (pour les appels réseau)

```
import time

def fetch_with_retry(url, max_retries=3):
    for attempt in range(max_retries):
        try:
            response = requests.get(url, timeout=30)
            response.raise_for_status()
            return response.json()
        except requests.RequestException as e:
            wait = 2 ** attempt
            logger.warning(f"Tentative {attempt+1} "
                           f"échouée. Retry dans {wait}s")
            time.sleep(wait)
    raise RuntimeError(f"Échec après {max_retries} tentatives")
```

TRAITEMENT PAR CHUNKS (pour gros volumes)

```
def process_large_file(filepath, chunk_size=10000):
    results = []
    for chunk in pd.read_csv(filepath,
                             chunksize=chunk_size):
        processed = transform(chunk)
        results.append(processed)
        logger.info(f"{len(results)*chunk_size} lignes traitées")
    return pd.concat(results, ignore_index=True)
```

CONTEXT MANAGER (pour ressources à fermer)

```
# Toujours utiliser with pour fichiers et connexions
with open("data.csv", "r", encoding="utf-8") as f:
    data = f.read()
```

```
with engine.connect() as conn:
    df = pd.read_sql(query, conn)
```

VÉRIFICATION DE SCHÉMA EN ENTRÉE

```
REQUIRED_COLUMNS = {"id", "date", "montant", "statut"}

def validate_input(df):
    missing = REQUIRED_COLUMNS - set(df.columns)
    if missing:
        raise ValueError(f"Colonnes manquantes : {missing}")
    if df.empty:
        raise ValueError("Le DataFrame d'entrée est vide")
    return True
```

"Un script qui échoue silencieusement est pire
qu'un script qui n'existe pas."

3.1 SETUP LOGGING AVEC LOGURU (recommandé)

```
from loguru import logger

logger.add(
    "logs/pipeline_{time:YYYY-MM-DD}.log",
    level="INFO",
    rotation="1 day",
    retention="30 days",
    format="{time:HH:mm:ss} | {level} | {message}"
)

# Niveaux à utiliser :
logger.debug("Détail technique de développement")
logger.info("Étape franchise, volume traité")
logger.warning("Anomalie tolérée, traitement continue")
logger.error("Erreur récupérée, impact partiel")
logger.critical("Arrêt du script, intervention requise")
logger.success("Étape ou pipeline terminé avec succès")
```

3.2 CE QU'ON LOGUE TOUJOURS

- Début et fin de chaque étape majeure
- Nombre de lignes à chaque transition
- Paramètres de configuration utilisés (sans credentials)
- Chaque exception catchée avec son contexte
- Durée d'exécution des étapes longues
- Résultat de validation post-traitement

3.3 HIÉRARCHIE DES ERREURS – QUOI FAIRE DANS CHAQUE CAS

ERREUR FATALE → arrêter proprement

Exemples : fichier source introuvable, connexion BDD impossible, schéma invalide
Action : logger.critical() + sys.exit(1)
+ notifier si en production

ERREUR RÉCUPÉRABLE → loguer, continuer

Exemples : une ligne malformée parmi 10 000, un retry réussi, une valeur manquante connue et acceptable
Action : logger.warning() + compteur d'anomalies
+ signaler dans le résumé final

ERREUR SILENCIEUSE → la pire catégorie

Exemples : un except: pass, une valeur remplacée sans log, une jointure qui perd des lignes
Action : JAMAIS. Toujours loguer ce qui est dévié.

3.4 RÉSUMÉ D'EXÉCUTION EN FIN DE SCRIPT

- Toujours finir par un récapitulatif :

```
logger.info("=== RÉSUMÉ ===")
logger.info(f"Lignes extraites : {n_extracted}")
logger.info(f"Lignes valides : {n_valid}")
logger.info(f"Lignes rejetées : {n_rejected}")
logger.info(f"Lignes chargées : {n_loaded}")
logger.info(f"Durée totale : {elapsed:.1f}s")
if n_rejected > 0:
    logger.warning(f"{n_rejected} lignes rejetées – vérifier logs")
```

PHASE 4 – TESTS

"Un code non testé est un code qui fonctionne jusqu'au premier vrai usage."

4.1 CE QU'ON TESTE EN PRIORITÉ

- Les règles de transformation (le cœur du métier)
- Les validations de schéma

- Les cas limites : fichier vide, valeurs nulles, formats inattendus, données hors plage
- Le comportement en cas d'erreur attendue

On ne teste PAS en priorité :

- Les appels réseau (on les mocke)
- Les lectures/écritures fichiers simples
- Le code qui change souvent

4.2 STRUCTURE D'UN TEST UNITAIRE

```
# tests/test_transform.py
import pytest
import pandas as pd
from src.transform import clean_dates, filter_valid

def test_clean_dates_format_standard():
    df = pd.DataFrame({"date": ["2024-01-15"]})
    result = clean_dates(df)
    assert result["date"].dtype == "datetime64[ns]"

def test_clean_dates_valeur_invalide():
    df = pd.DataFrame({"date": ["pas-une-date"]})
    result = clean_dates(df)
    assert len(result) == 0 # ligne rejetée

def test_filter_valid_dataframe_vide():
    df = pd.DataFrame(columns=["id", "montant"])
    with pytest.raises(ValueError):
        filter_valid(df)

Lancer les tests :
pytest tests/ -v
```

4.3 NIVEAU DE COUVERTURE ATTENDU PAR TYPE DE PROJET

```
Script one-shot    → 0% (tester manuellement suffit)
Script récurrent   → tester les fonctions critiques
Outil interne      → > 60% de couverture
Produit livré      → > 80% de couverture + CI
```

PHASE 5 – DÉPLOIEMENT & PLANIFICATION

5.1 CHECKLIST AVANT DÉPLOIEMENT EN PRODUCTION

```
[ ] Le script tourne sans erreur sur données réelles
[ ] Les credentials sont dans .env, pas dans le code
[ ] Le logging est activé et les logs sont persistés
[ ] Un résumé d'exécution est produit à chaque run
[ ] Le comportement sur données vides est géré
[ ] La documentation d'installation est à jour
[ ] Le client sait comment surveiller les logs
[ ] Un contact de maintenance est défini
```

5.2 OPTIONS DE PLANIFICATION

LINUX – CRON

```
crontab -e
```

```
# Chaque jour à 7h00
```

```
0 7 * * * /chemin/venv/bin/python /chemin/main.py
```

```
# Toutes les heures
```

```
0 * * * * /chemin/venv/bin/python /chemin/main.py
```

```
# Rediriger les logs cron
```

```
0 7 * * * /chemin/venv/bin/python /chemin/main.py >> /chemin/logs/cron.log 2>&1
```

WINDOWS – TASK SCHEDULER

```
→ Ouvrir "Planificateur de tâches"
```

```
→ Nouvelle tâche → Action : lancer programme
```

```
→ Programme : C:\projet\venv\Scripts\python.exe
```

→ Arguments : C:\projet\main.py

CLOUD – OPTIONS LÉGÈRES

GitHub Actions → déclencheur cron dans le YAML
PythonAnywhere → scheduler intégré, gratuit
AWS Lambda → EventBridge pour le scheduling
GCP Cloud Run → Cloud Scheduler

5.3 SURVEILLER UN SCRIPT EN PRODUCTION

SIGNAUX D'ALERTE À MONITORER :

→ Volume de sortie anormalement bas ou nul
→ Durée d'exécution anormalement longue
→ Taux d'erreurs / rejets en hausse
→ Échec complet sans notification

NOTIFICATION EN CAS D'ÉCHEC (email simple)

```
import smtplib
from email.mime.text import MIMEText

def notify_failure(error_msg: str):
    msg = MIMEText(f"Pipeline échoué :\n{error_msg}")
    msg["Subject"] = "ALERTE – Script en échec"
    msg["From"] = os.getenv("SMTP_FROM")
    msg["To"] = os.getenv("SMTP_TO")
    with smtplib.SMTP(os.getenv("SMTP_HOST"), 587) as s:
        s.starttls()
        s.login(os.getenv("SMTP_USER"),
                os.getenv("SMTP_PASS"))
        s.send_message(msg)
```

PHASE 6 – DOCUMENTATION & LIVRAISON

6.1 README.md MINIMAL OBLIGATOIRE

```
# Nom du projet
Description courte en 2 lignes.

## Prérequis
- Python 3.10+
- Accès à [système X]

## Installation
git clone ...
cd projet
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # remplir les valeurs

## Configuration
| Variable      | Description                | Exemple      |
|-----|-----|-----|
| DB_HOST       | Hôte base de données      | localhost    |
| API_KEY       | Clé API du service X      | abc123       |

## Utilisation
python main.py

## Logs
Les logs sont générés dans logs/

## En cas de problème
Contacter : [ton nom] – [ton email]
```

6.2 COMMENTAIRES DANS LE CODE

```
COMMENTER LE POURQUOI, PAS LE QUOI
# Mauvais :
# On filtre les lignes
df = df[df["statut"] == "actif"]
```

```
# Bon :
# Exclure les comptes désactivés : ils faussent
# les moyennes depuis la migration de mai 2024
df = df[df["statut"] == "actif"]
```

6.3 CHECKLIST LIVRAISON FINALE

- [] README.md complet et testé (installation fraîche)
- [] .env.example avec toutes les variables documentées
- [] requirements.txt à jour avec versions figées
- [] Logs de la dernière exécution réussie conservés
- [] Formation client faite (ou enregistrée)
- [] Accès au repo transmis
- [] Contrat de maintenance ou clause de fin de mission

RÉFÉRENCE RAPIDE – LIBRAIRIES PAR USAGE

DONNÉES & TRAITEMENT

pandas → manipulation de tableaux
 polars → pandas ultra-rapide sur gros volumes
 numpy → calcul numérique
 openpyxl → lire/écrire Excel
 pdfplumber → extraire texte/tables de PDF

BASE DE DONNÉES

sqlalchemy → ORM et connexion universelle
 psycopg2 → PostgreSQL
 pymysql → MySQL
 sqlite3 → SQLite (inclus dans Python)

API & WEB

requests → appels HTTP
 httpx → requests async
 BeautifulSoup4 → parsing HTML
 selenium → automatisation navigateur
 playwright → selenium moderne (recommandé)

AUTOMATISATION SYSTÈME

pathlib → manipulation de chemins (inclus)
 shutil → copie/déplacement de fichiers (inclus)
 subprocess → lancer des commandes système (inclus)
 schedule → planification légère en pur Python
 watchdog → surveiller des changements de fichiers

EMAIL & NOTIFICATIONS

smtplib → email SMTP (inclus)
 yagmail → envoi Gmail simplifié
 slack-sdk → notifications Slack
 python-telegram-bot → bot Telegram

LOGGING & CONFIG

loguru → logging moderne et simple
 python-dotenv → gestion .env
 pydantic → validation de config et de données
 click → arguments en ligne de commande

TESTS

pytest → framework de tests
 pytest-cov → couverture de code
 responses → mock d'appels HTTP dans les tests

CHECKLIST UNIVERSELLE – À APPLIQUER SUR TOUT PROJET

CONCEPTION

- [] Le problème est formulé en une phrase claire
- [] Le flux est décomposé en étapes indépendantes
- [] Les cas d'erreur sont identifiés avant de coder
- [] La complexité choisie est adaptée au besoin réel

STRUCTURE

- [] Environnement virtuel activé
- [] .env en place et dans .gitignore
- [] config.py centralisé, aucune valeur magique dans le code
- [] Structure de dossiers cohérente avec la complexité

CODE

- [] Une fonction = une responsabilité
- [] Type hints sur les fonctions critiques
- [] Aucun credential en dur
- [] Context managers pour les ressources
- [] Validation du schéma d'entrée

ROBUSTESSE

- [] Logging activé sur toutes les étapes
- [] Résumé d'exécution produit en fin de run
- [] Erreurs fatales → exit propre avec message
- [] Erreurs tolérables → log + continue + compteur
- [] Aucun except sans log

DÉPLOIEMENT

- [] Tourne sans erreur sur données réelles
- [] Planification configurée et testée
- [] Notification en cas d'échec critique
- [] README installable par quelqu'un d'autre

LIVRAISON

- [] requirements.txt à jour avec versions
- [] Documentation à jour
- [] Client formé et autonome (ou maintenance contractualisée)

VÉRITÉS FINALES SUR L'AUTOMATISATION EN PYTHON

- La première version doit fonctionner, pas être parfaite.
Refactorise après, pas avant.
 - Un script sans logging en production n'existe pas.
Quand il plantera, tu ne sauras pas pourquoi.
 - Les cas limites ne sont pas des cas rares.
En production, tout ce qui peut arriver arrive.
 - Le code le plus facile à maintenir est celui
qu'on peut lire six mois plus tard sans se souvenir
de l'avoir écrit.
 - Automatiser une mauvaise procédure ne la corrige pas.
Elle la reproduit plus vite, à plus grande échelle.
 - "Ça marche sur mon poste" n'est pas une livraison.
Une livraison, c'est quand le client le fait tourner
seul, la première fois, sans t'appeler.
-